

EarnKaro Price Comparison Telegram Bot — Claude Code Task List

Platform: Google Colab | Stack: Python, BeautifulSoup, requests, python-telegram-bot, gspread

Project Structure (Final)

```
earnkaro_bot/
├── bot.py           # Main Telegram bot entry point
├── extractor.py     # Scrapes source product URL → structured data
├── searcher.py      # Searches each platform, returns results
├── matcher.py       # Scores and picks best match per platform
├── sheets.py        # Google Sheets commission lookup
├── formatter.py     # Builds the Telegram message
├── config.py        # All constants, headers, CSS selectors
├── utils.py         # Shared helpers (quantity normalizer, etc.)
└── requirements.txt # All dependencies
```

TASK 1 — Project Setup & Dependencies

1.1 Create `requirements.txt`

```
requests==2.31.0
beautifulsoup4==4.12.2
lxml==4.9.3
rapidfuzz==3.5.2
python-telegram-bot==20.6
gspread==5.12.0
google-auth==2.23.4
fake-useragent==1.4.0
```

1.2 Create Colab Setup Cell

Create a code cell at the top of the Colab notebook that:

- Runs `pip install -r requirements.txt`
- Mounts Google Drive at `/content/drive` using `drive.mount()`
- Sets working directory to `/content/drive/MyDrive/earnkaro_bot/`
- Prints "✅ Setup complete" when done

1.3 Create `config.py`

Define the following constants:

```
python

# Platform domain detection
PLATFORM_DOMAINS = {
    "amazon.in": "amazon",
    "flipkart.com": "flipkart",
    "myntra.com": "myntra",
    "hamaramall.com": "hamaramall"
}

# Request headers per platform
HEADERS = {
    "amazon": {
        "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64)...",
        "Accept-Language": "en-IN,en;q=0.9",
    },
    "flipkart": {
        "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64)...",
    },
    "myntra": {
        "User-Agent": "Mozilla/5.0 ...",
    },
    "hamaramall": {
        "User-Agent": "Mozilla/5.0 ...",
    }
}

# Matching score thresholds
SCORE_EXACT = 75      # Show as  Exact match
SCORE_SIMILAR = 40    # Show as  Similar product
# Below 40 =  Not found

# Search result count to evaluate per platform
TOP_N_RESULTS = 5

# Delays between requests (seconds) to avoid rate limiting
REQUEST_DELAY = 1.5

# Google Sheet config
SHEET_NAME = "EarnKaro Commissions"
SHEET_TAB = "Category Commission"
COMMISSION_COL_CATEGORY = 0    # Column A
COMMISSION_COL_PERCENT = 1     # Column B
```

TASK 2 — `utils.py` (Shared Helpers)

2.1 `detect_platform(url: str) -> str`

- Parse the URL domain
- Match against `PLATFORM_DOMAINS` in config
- Return platform name string e.g. `"amazon"`, `"flipkart"`
- Raise `ValueError` if domain not recognised
- Test with: amazon.in, flipkart.com, myntra.com, hamaramall.com URLs

2.2 `normalize_quantity(text: str) -> str`

Standardize quantity strings for comparison:

- Strip all spaces: `"250 ml"` → `"250ml"`
- Lowercase: `"250ML"` → `"250ml"`
- Handle weight: `"1 KG"` → `"1000g"` (convert kg→g for comparison)
- Handle count: `"pack of 3"` → `"3pack"`
- Return normalized string

2.3 `extract_quantity_from_title(title: str) -> str`

- Use regex to find quantity patterns in a product title
- Patterns to catch: `250ml`, `500 ml`, `1kg`, `200g`, `100gm`, `pack of 2`, `x3`
- Return the matched quantity string or `""` if none found

2.4 `clean_price(price_str: str) -> int`

- Take a raw price string like `"₹1,299"` or `"Rs. 1299.00"`
- Strip currency symbols, commas, decimals
- Return integer e.g. `1299`
- Return `0` if price cannot be parsed

2.5 `extract_keywords(title: str, brand: str) -> list`

- Take full product title and brand name
 - Remove brand name from title
 - Remove stop words: `["for", "with", "and", "of", "the", "in", "a", "an", "-"]`
 - Remove quantity strings (use `extract_quantity_from_title`)
 - Return list of meaningful keywords e.g. `["onion", "hair", "oil", "growth", "control"]`
-

TASK 3 — `extractor.py` (Source Page Scraper)

This is the most important file. Given the user's product URL, extract structured data.

3.1 Main function signature

```
python

def extract_product(url: str) -> dict:
    # Returns:
    {
        "platform": "amazon",
        "url": "https://...",
        "brand": "Mamaearth",
        "name": "Onion Hair Oil for Hair Growth & Hair Fall Control",
        "quantity": "250ml",
        "keywords": ["onion", "hair", "oil", "growth"],
        "category": "Hair Care",
        "price": 299,
        "search_query": "Mamaearth Onion Hair Oil 250ml"
    }
```

3.2 `extract_from_amazon(soup, url) -> dict`

Scrape an Amazon product page:

- **Brand:** `#bylineInfo` span, or first word of title before first space
- **Title:** `#productTitle` → `.strip()`
- **Price:** Try `#priceblock_ourprice` → then `.a-price-whole` → then `#priceblock_dealprice`
- **Category:** Breadcrumb `#wayfinding-breadcrumbs_feature_div` → second-to-last item
- **Quantity:** Run `extract_quantity_from_title()` on the title

3.3 `extract_from_flipkart(soup, url) -> dict`

Scrape a Flipkart product page:

- **Brand:** Breadcrumb second item, or first word of `._35KyD6` title class
- **Title:** `._35KyD6` or `B_NuCI` class → inspect current Flipkart HTML to confirm
- **Price:** `._30jeq3._16Jk6d` class → clean with `clean_price()`
- **Category:** Breadcrumb `._2whKao` items → second-to-last
- **Quantity:** `extract_quantity_from_title()` on title

3.4 `extract_from_myntra(soup, url) -> dict`

Scrape a Myntra product page:

- **Brand:** `.pdp-title` class (brand is usually the first line)
- **Title:** `.pdp-name` class
- **Price:** `.pdp-price strong` or `.pdp-mrp`
- **Category:** URL slug — Myntra URLs follow pattern `/brand/product/category/` — parse from URL path
- **Quantity:** `extract_quantity_from_title()` on title

3.5 `extract_from_hamaramall(soup, url) -> dict`

Scrape a Hamara Mall product page:

- Inspect live Hamara Mall product page HTML to identify correct selectors
- Extract: brand, title, price, category from page
- **Note:** Document the actual CSS selectors found during inspection

3.6 `build_search_query(product: dict) -> str`

- Combine `brand + name_core + quantity`
- Name core = title with stop words and quantity already removed
- Trim to max 60 characters
- Return clean search string e.g. `"Mamaearth Onion Hair Oil 250ml"`

3.7 Error handling in `extract_product()`

- Wrap all extraction in try/except
- If a field can't be found, set it to `""` or `0`
- Never crash — return partial data with a `"warning"` key listing missing fields
- Log which selectors failed for debugging

TASK 4 — `searcher.py` (Cross-Platform Search)

4.1 `search_amazon(query: str) -> list`

- URL: `https://www.amazon.in/s?k={query}`
- Use `requests.get()` with Amazon headers from config + `time.sleep(REQUEST_DELAY)`
- Parse with BeautifulSoup
- For each result in `div[data-component-type="s-search-result"]`:
 - Extract: title (`.a-size-medium` or `.a-size-base-plus`), price (`.a-price-whole`), URL (`a.a-link-normal`)
- Return list of dicts: `[{"title": ..., "price": ..., "url": ...}]`
- Return max `TOP_N_RESULTS` items

- Return `[]` on any error

4.2 `search_flipkart(query: str) -> list`

- URL: `https://www.flipkart.com/search?q={query}`
- Use `requests.get()` with Flipkart headers
- Parse search results — inspect current Flipkart search HTML for correct selectors
- Extract: title, price, URL per result
- Return list of dicts, max `TOP_N_RESULTS`
- **Note:** If requests returns empty results (JS-rendered), flag for Selenium fallback

4.3 `search_myntra(query: str) -> list`

- URL: `https://www.myntra.com/{query}` (Myntra search uses URL path)
- Use `requests.get()` with Myntra headers
- Inspect response — Myntra may return JSON in a `<script>` tag
- If JSON found: parse `__INITIAL_STATE__` or similar key for product list
- If HTML: parse `.product-base` or `.results-base` classes
- Return list of dicts, max `TOP_N_RESULTS`

4.4 `search_hamaramall(query: str) -> list`

- Inspect Hamara Mall's search URL pattern first
- Likely: `https://www.hamaramall.com/search?q={query}`
- Parse results with BeautifulSoup
- Return list of dicts, max `TOP_N_RESULTS`

4.5 `search_all_platforms(product: dict, skip_platform: str) -> dict`

- Takes the extracted product dict and the source platform name
- Runs search on all platforms EXCEPT `skip_platform` (the source)
- Runs searches with `time.sleep(REQUEST_DELAY)` between each
- Returns:

```
python
{
    "amazon": [...results...],
    "flipkart": [...results...],
    "myntra": [...results...],
    "hamaramall": [...results...]
}
```

TASK 5 — `matcher.py` (Scoring & Match Selection)

5.1 `score_result(source: dict, result: dict) -> int`

Score a single search result against the source product (max 100):

```
python

score = 0

# Rule 1 – Brand match (required gate, +40)
# If brand not found anywhere in result title → return 0 immediately
if source["brand"].lower() not in result["title"].lower():
    return 0
score += 40

# Rule 2 – Quantity match (+35)
# Normalize both, compare
source_qty = normalize_quantity(source["quantity"])
result_qty = extract_quantity_from_title(result["title"])
if source_qty and result_qty and source_qty == normalize_quantity(result_qty):
    score += 35

# Rule 3 – Keyword overlap (+25)
# What % of source keywords appear in result title
source_kw = set(source["keywords"])
result_words = set(result["title"].lower().split())
if source_kw:
    overlap = len(source_kw & result_words) / len(source_kw)
    score += int(overlap * 25)

return score
```

5.2 `pick_best_match(source: dict, results: list) -> dict`

- Score all results using `score_result()`
- Sort by score descending
- Return best result with its score attached: `{...result, "score": 82, "match_type": "exact"}`
- Set `match_type`:
 - `score >= SCORE_EXACT` → `"exact"`
 - `score >= SCORE_SIMILAR` → `"similar"`
 - `score < SCORE_SIMILAR` → `"not_found"`

5.3 `fallback_search(source: dict, platform: str) -> dict`

Called when best score is `"not_found"`:

- **Fallback 1:** Drop quantity from query, search `"brand + product name"` only
- **Fallback 2:** Drop brand too, search `"product type + key ingredient"` e.g. `"onion hair oil"`
- Re-run the appropriate search function for the platform
- Score results again
- If still below threshold, return `{"match_type": "unavailable"}`

5.4 `match_all_platforms(source: dict, search_results: dict) -> dict`

- For each platform in `search_results`:
 - Run `pick_best_match()`
 - If `"not_found"`, run `fallback_search()`
- Return:

```
python
{
    "amazon":      {"title": ..., "price": 299, "url": ..., "match_type": "exact"},
    "flipkart":    {"title": ..., "price": 319, "url": ..., "match_type": "exact"},
    "myntra":      {"title": ..., "price": 349, "url": ..., "match_type": "similar"},
    "hamaramall":  {"match_type": "unavailable"}
}
```

TASK 6 — `sheets.py` (Commission Lookup)

6.1 Google Sheets Auth Setup in Colab

- Create a Colab cell that:
 - Prompts user to upload `service_account.json` (Google Cloud service account key)
 - Saves it to `/content/drive/MyDrive/earnkaro_bot/service_account.json`
 - Prints instructions: share the Google Sheet with the service account email

6.2 `load_commission_sheet() -> dict`

- Authenticate using `gspread` with service account JSON
- Open sheet by name `SHEET_NAME` from config
- Read all rows from tab `SHEET_TAB`

- Build and return a dict: `{"Hair Care": 12.0, "Skin Care": 10.0, ...}`
- Cache result in a module-level variable so it's only fetched once per session

6.3 `get_commission(category: str, commission_map: dict) -> float`

- Exact match first: look up `category` directly
- If not found: try case-insensitive match
- If not found: try partial match (e.g. `"Hair"` matches `"Hair Care"`)
- Return commission % as float e.g. `12.0`
- Return `0.0` if no match found

6.4 `refresh_commission_cache()`

- Clear the module-level cache variable
- Re-call `load_commission_sheet()`
- Use this if commissions are updated mid-session

TASK 7 — `formatter.py` (Telegram Message Builder)

7.1 `format_platform_row(platform: str, match: dict, source_platform: str) -> str`

Build one line of the price comparison table:

	Amazon	₹299		# exact match
	Flipkart	₹319		
	Myntra	₹349		similar
	Hamara Mall	—		not listed

- Platform emoji map: Amazon , Flipkart , Myntra , Hamara Mall 
- Source platform row gets `(your link)` appended
- Prices right-aligned using padding
- Match type shown as  /  similar / 

7.2 `find_best_deal(matches: dict) -> tuple`

- Look at all `"exact"` match prices
- Find lowest price platform
- Find highest commission earning (price × commission%)
- Return `(cheapest_platform, best_earning_platform)`

7.3 `format_full_message(source: dict, matches: dict, commission: float) -> str`

Assemble the complete Telegram message:

```
{Brand} {Product Name} {Quantity}
{Category} | 🏪 EarnKaro Commission: {commission}%

💰 Price Comparison


---


{platform rows}

🏆 Lowest price: {platform} ₹{price}
💡 Best earning: {platform} → ₹{earning}

⚠️ Note: Similar products may differ in size/variant
```

- Only show the ⚠️ note if any platform has `match_type == "similar"`
- If commission is 0.0, omit the earning line

TASK 8 — `bot.py` (Telegram Bot — Main Entry Point)

8.1 Bot Initialization Cell in Colab

- Create a Colab cell to:
 - Prompt for `TELEGRAM_BOT_TOKEN` using `getpass` (hidden input)
 - Store in environment variable
 - Print "Token saved ✅"

8.2 `handle_message(update, context)` — Main Handler

This runs every time a user sends a message to the bot:

```
python
```

```

async def handle_message(update, context):
    url = update.message.text.strip()

    # Step 1 - Validate it's a known platform URL
    platform = detect_platform(url)
    if not platform:
        await update.message.reply_text("⚠️ Please send a product link from Amazon,
        return

    # Step 2 - Acknowledge immediately (user sees response fast)
    await update.message.reply_text("⌚ Searching across platforms, give me a moment

    # Step 3 - Extract product from source URL
    product = extract_product(url)

    # Step 4 - Search all other platforms
    search_results = search_all_platforms(product, skip_platform=platform)

    # Step 5 - Match and score
    matches = match_all_platforms(product, search_results)

    # Step 6 - Get commission
    commission = get_commission(product["category"], commission_map)

    # Step 7 - Format and send
    message = format_full_message(product, matches, commission)
    await update.message.reply_text(message, parse_mode="HTML")

```

8.3 `/start` Command Handler

Reply with:

👋 Welcome to EarnKaro Price Bot!

Send me any product link from:

- Amazon.in
- Flipkart
- Myntra
- Hamara Mall

I'll compare prices across all platforms and show your EarnKaro commission instantly.

8.4 Error Handler

- Catch all unhandled exceptions in `handle_message`
- Reply to user: `"❌ Something went wrong. Please try again or send a different link."`
- Print full traceback to Colab console for debugging

8.5 Bot Startup Cell

Final Colab cell that:

- Loads commission map: `commission_map = load_commission_sheet()`
 - Initializes `Application` with token
 - Registers handlers for `/start` and all messages
 - Starts polling with `application.run_polling()`
 - Prints `"🤖 Bot is running... Send it a product link on Telegram"`
-

TASK 9 — Testing

9.1 Unit Test Cell — `utils.py`

Test each utility function with hardcoded inputs:

```
python

assert normalize_quantity("250 ML") == "250ml"
assert normalize_quantity("1 KG") == "1000g"
assert clean_price("₹1,299") == 1299
assert extract_quantity_from_title("Mamaearth Onion Hair Oil 250ml") == "250ml"
```

9.2 Unit Test Cell — `extractor.py`

Test with 5 real product URLs (one per platform + one edge case):

- Paste 5 real beauty/grocery product URLs
- Print extracted dict for each
- Manually verify brand, quantity, category are correct

9.3 Unit Test Cell — `matcher.py`

Test scoring with mock data:

```
python
```

```
source = {"brand": "Mamaearth", "quantity": "250ml", "keywords": ["onion", "hair", "  
result_exact = {"title": "Mamaearth Onion Hair Oil 250ml", "price": 299}  
result_similar = {"title": "Mamaearth Onion Hair Oil 100ml", "price": 199}  
result_different = {"title": "WOW Onion Hair Oil 250ml", "price": 280}  
  
assert score_result(source, result_exact) >= 75  
assert 40 <= score_result(source, result_similar) < 75  
assert score_result(source, result_different) == 0 # brand mismatch
```

9.4 End-to-End Test Cell

Pick 3 real product URLs. For each:

- Run the full pipeline: extract → search → match → commission → format
- Print the final formatted message
- Verify it looks correct before connecting the Telegram bot

TASK 10 — Colab Keep-Alive & Session Management

10.1 Keep-Alive Cell

Google Colab disconnects after ~90 minutes of inactivity. Add a cell using `javascript` to click the connect button periodically:

```
javascript  
  
// Run this cell to prevent Colab timeout  
function ClickConnect(){  
    console.log("Keeping alive...");  
    document.querySelector("colab-toolbar-button#connect").click()  
}  
setInterval(ClickConnect, 60000)
```

10.2 Session Restart Instructions

Add a markdown cell with instructions for when the session restarts:

1. Run Cell 1: pip install
 2. Run Cell 2: mount Drive
 3. Run Cell 3: load commission sheet
 4. Run Cell 4: enter Telegram token
 5. Run Cell 5: start bot
-

Known Gotchas — Read Before Building

- **Flipkart CSS classes change frequently** — store all selectors in `config.py` so updates are in one place
- **Amazon blocks repeated requests** — always use `time.sleep(REQUEST_DELAY)` and rotate User-Agent using `fake-useragent`
- **Myntra search may return JSON in a script tag** — parse `window.__INITIAL_STATE__` or similar
- **Quantity in different formats** — `"200 ml"`, `"200ml"`, `"200 ML"` must all match; `normalize_quantity()` handles this
- **Hamara Mall selectors unknown** — manually inspect a live product page before writing that extractor
- **Google Colab session dies** — commission map must be reloaded on restart; keep-alive cell helps but isn't perfect
- **Telegram polling blocks** — `run_polling()` must be the last cell; can't run other cells while it's running